UCLouvain

epl

École polytechnique de Louvain

Toward verification of QUIC (extensions)

Use of an automated tool for a formal verification of the QUIC protocol (and extensions) : Ivy

Authors: **Christophe Crochet, Jean-François Sambon**
Supervisors: **Olivier Bonaventure, Axel Legay**
Readers: **Charles Pêcheur, Maxime Piraux**
Academic year 2020–2021
Master [120] in Computer Science

Acknowledgements

Crochet Christophe

Sambon Jean-Francois

Abstract

Glossary

Ack-eliciting Packet "A QUIC packet that contains frames other than ACK, PADDING, and CONNECTION_CLOSE. These cause a recipient to send an acknowledgment; see Section 13.2.1". ii

Connection ID "An opaque identifier that is used to identify a QUIC connection at an endpoint. Each endpoint sets a value for its peer to include in packets sent towards the endpoint.". ii

Out-of-order packet "A packet that does not increase the largest received packet number for its packet number space (Section 12.3) by exactly one. A packet can arrive out of order if it is delayed, if earlier packets are lost or delayed, or if the sender intentionally skips a packet number". ii

Probing packet "PATH_CHALLENGE, PATH_RESPONSE, NEW_CONNECTION_ID, and PADDING frames are "probing frames", and all other frames are "non-probing frames". A packet containing only probing frames is a "probing packet", and a packet containing any other frame is a "non-probing packet". ". ii

QUIC stands for "Quick UDP Internet Connections". ii

QUIC Frame "The payload of QUIC packets, after removing packet protection, consists of a sequence of complete frames, as shown in Figure 10. Version Negotiation, Stateless Reset, and Retry packets do not contain frames.". ii

QUIC Packet "QUIC endpoints communicate by exchanging packets. Packets have confidentiality and integrity protection; see Section 12.1. Packets are carried in UDP datagrams;". ii

Stream "A unidirectional or bidirectional channel of ordered bytes within a QUIC connection. A QUIC connection can carry multiple simultaneous streams.". ii

TLS stands for "Quick UDP Internet Connections". ii

UDP stands for "Quick UDP Internet Connections". ii

Contents

List of Figures

List of Tables

Listings

Chapter 1

Results

1.1 Introduction

In this chapter, we show you the results we obtained by following the experimental protocol of McMillan's paper's "Formal Specification and Testing of QUIC" applied to the draft 29 [1] for the server and client implementation. We also compare our actual results with the preceding ones to see the evolution of the correctness of the different implementations.

doit on mettre plus de mots clé en gras ?

1.2 Methodology

1.2.1 Virtual Machine

We decide to launch our tests on a `CentOS (7.5.1804)` virtual machine running on an `x86_64` architecture with 4 cores of 3100 MHz and 8 Gb of RAM.

1.2.2 Limits to improve

Some points limited the coverage of our tests. First of all, originally, Ivy accepts variables of maximum 8 bytes which was not a problem for the implementations that McMillan tested. We manage to improve Ivy to support data up to 16 bytes. In the last chapter, we explore some paths to go beyond this problem. This maximal value of 16 bytes for Ivy datatype limits some tests we could do. It is also a limitation to generate a specific scenario for some implementation. One example is the Connection ID. The draft says that it can be between 0 and 20 bytes but only a subset of implementations use a Connection ID of more than 16 bytes. This is a restriction if there are more than 16 bytes that are used because we would have to truncate the value and that could cause problems for part of our tests. When we had this issue, most of the time we solved it by modifying the implementations themselves so they take connection ID of maximum 16 bytes. Generally, it consisted of modifying just one global variable.

Another point is that we could have done more tests for some errors but we considered that it would have been repetition to create tons of similar tests. It would have significantly increased the time needed to launch all the tests without a great improvement in the quality of our results. For example, concerning errors that are linked to transport parameters but not a specific one, we focus our tests by choosing an arbitrary value for a single transport parameter.

We have also seen some cases where the computational time of Ivy was causing a problem, especially when an implementation sends a batch of big packets in a very short time. This slows down Ivy and can cause the tested implementation to timeout its retransmissions timer. It could result on some false positive. A case of this problem is with `quic-go` that typically sends us a batch of 5 streams frames and timeout. You can check the results for this implementation for more details.

1.2.3 Implementation tested

Concerning Ivy, we checkout the `quic_29` branch of our Ivy forked version [2]. We decided to test seven QUIC implementations: `quinn`, `picoquic`, `aioquic`, `mvfst`, `quiche`, `quic-go` and `lsquic`. We select famous implementation made in different programming languages. Different TLS implementations are used among the implementations. Some of them are mature implementations that exist from a long time while others are quite new. We have implementations that are intended to be used in production and some that are used for scientific and feedback purposes. You can find more details in [3].

presenter un peu les implementation ?

picoquic (C) [4] ad23e6c3593bd987dcd8d74fc9f528f2676fedf4 **picotls** [5] 47327f8d032f6bc2093a15c32e666ab6384ecca2
quic-go (golang) [6] v0.18.1
aioquic (python) [7] 0.9.3
quant (C) [8] 29
mvfst (facebook/C++) [9] 36111c1
lsquic [10] v2.29.4 **boringssl** [11] a2278d4d2cabe73f6663e3299ea7808edfa306b9
quinn (rust) [12] 0.7.0
quiche (rust) [12] 0.7.0

Table 1.1: Draft 29 tested implementations

1.2.4 Our tests

Server tests

We have 14 different tests for the server part. Some of them are general while others are specific.

First, we have the original tests of McMillan. They are not the same as before because we had to adapt them to draft 29. There is `quic_server_test_stream` that is a general scenario for testing streams. During this test, we allow multiple migrations so we can see if the tested implementation can manage it. Several streams are opened to see if the management of streams is correctly done and we create them by increasing the stream ID by 4. We have a second test `quic_server_test_max` that is specific to the handling of the `max_stream_data` and `stream_data_blocked` frames. We test if the amount of data received on a stream is not over the `max_stream_data` limit. If it goes above the limit, we test if we receive well `stream_data_blocked frame`. The two last tests, `quic_server_test_reset_stream`, `quic_server_test_connection_close`, are testing respectively the receiving of a `RESET_STREAM` frame if the stream is well reset and the way that the tested implementation close his connection.

We also added ten more scenarios. The test `quic_server_test_unkown` checks the behavior of an implementation that receives a frame of an unknown type after the handshake has been done. We should receive a `FRAME_ENCODING_ERROR` according to the draft. Note that locally for this test, we allow this frame to be generated at any time to see how the implementation behaves if they receive this frame at the wrong encryption level. This won't be reflected in the result tables but we develop some points based on that.

There are several tests for transport parameters since this is one of the parts that changed the most between the two tested drafts. With `quic_server_test_tp_error`, we put a wrong value to one of the transport parameters and expect a `TRANSPORT_PARAMETER_ERROR`. We run this test for each transport parameter where there is at least one property to verify and at each run, we set a wrong value to a single transport parameter. We have one specific test for the `active_connection_id_limit` transport parameter in the test `quic_server_test_tp_acticoid_error` where we initiate a connection with this transport parameter value set at a value less than 2. We expect a `TRANSPORT_PARAMETER_ERROR`. We are also testing a scenario in `quic_server_test_no_icid` where we voluntary don't set the `initial_source_connection_id` transport parameter and check that the tested implementation throw a `TRANSPORT _PARAMETER_ERROR`. The test `quic_server_test_double_tp_error` is also about transport parameters. We set twice a transport parameter and expect a `TRANSPORT_PARAMETER_ERROR`. The rule in the draft is ambiguous as it is not clearly said what we should do in such a case. Should we trigger an error? Discard the connection? Something else? If a server accepts the connection, this can be problematic. For example, if we set twice the `max_ack_delay` and we decide to

not throw an error, which values to consider? Our interpretation in Ivy is that we should send an error since this is the most formal way of interpretation but the draft should be more precise. Here is the property:

> *An endpoint MUST NOT send a parameter more than once in a given transport parameters extension. An endpoint SHOULD treat receipt of duplicate transport parameters as a connection error of type* `TRANSPORT_PARAMETER_ERROR`.
>
> Draft 29 of QUIC section 7.4.

We have another test where we put a non-zero token field in an Initial packet. If a server receives such a packet, it must either discard the packet or either throw a `PROTOCOL_VIOLATION` error. We consider that there is no error from the tested implementation if we received a `PROTOCOL_VIOLATION` error or if the handshake never complete. In `quic_server_test_handshake_done_error`, we send a huge amount of `handshake_ done` frame at any time. According to the draft, a server should send this frame only before completing the handshake or return a `PROTOCOL_VIOLATION` error. This frame should only be sent by the server. Whenever we send this frame as a client, we should receive `PROTOCOL_VIOLATION` according to section [19.20.].

We also have one test about the handling of `STREAM_BLOCKED` frame. We generate this frame with an invalid "maximum stream" field value. The draft states that upon receiving that kind of frame, the server should send a `STREAM_LIMIT_ERROR` or `FRAME_ENCODING_ERROR`. We verify if this is the case.

There is another test that is similar to the test that was already done by McMillan `quic_server_test_max` except that we restrict to the `MAX_DATA` frame. There are also more value allowed in frame's fields domain. The coverage of the test is also higher than `quic_server_test_max`.

Finally, there is the `quic_server_test_ ext_min_ack_delay` that hold all the properties that we test of the `min_ack_delay` QUIC extension. Note that we didn't test the core properties of the extension since the nature itself about delay and time make it difficult to model. However, this test was more here to show that we can easily add new frames and new behavior for the extensions and that depending on the extension, more or fewer properties can be represented.

Let's note that in all those scenarios, we are keeping our model complete and all the rules that we implemented should be respected.

Client tests

McMillan initially made one test similar to his server counterpart. It is `quic_client_test_max` where the scenario is similar to `quic_server_test_max`. We added 9 other tests. Most of them are very similar to the server side : the scenario is the same. It is the case for the following tests : `quic_client_test_unkown`, `quic_client _test_tp_limit_acticoid_error`, `quic_client_test_tp_error`,

`quic_client_test_tp_acticoid_error`, `quic_client_test_no_odci`, `quic_client_test_handshake_done_error`,`quic_client_test_double_tp_error`, `quic_client_test_ext_min_ack_delay`.

There is one test specific to the client side, it is the `quic_client_test_tp_prefadd_error`. We test the fact that the `prefered_address` transport parameter should not contain zero-length connection ID and throw `TRANSPORT_PARAMETER_ERROR`.

Note that for the client part, we never perform connection migration.

1.3 Results

For this experience, we launch 100 iterations for each test and implementation. It took us 3 days to execute all the tests. A larger set of tests is possible but we don't think it would improve by a large factor the quality of our results. We base this observation on some tests that we launched at a large scale (1000+ tests) with no significant difference in the results. A batch of 100 tests seems sufficient to us. We should be careful about how we infer our results with statistical models. We preferred to focus on the analysis of traces than statistical analysis. We prefer to answer the question "how is it possible to improve an implementation" than "does my implementations have a good coverage of the RFC properties". Ivy should be able to answer both questions. However, for the second one, a deeper statistical analysis should be done. The way we proceed doesn't allow us to extract "probability" and confidence interval of whether an implementation respects a large part of the RFC or not. It should be possible to do statistical analysis but the different parameters of our experiment protocol should be review.

1.3.1 Servers results

We present in this section the server results. First, we present the result of `quant` and `picoquic`. As those two implementations were the ones that McMillan tested for draft 18, we can make a comparison between the previous and actual results. Then, we move on to the presentations of the results for the 7 other implementations. There is one implementation where no test passed `lsquic`. We end by explaining why we couldn't operate Ivy on it.
Here is a first overview of the results:

quinn	1428/1600
mvfst	549/1600
picoquic	919/1600
quic-go	500/1600
aioquic	584/1600
quant	320/1600
quiche	862/1600
lsquic	0/1600

Table 1.2: Draft 29 server results overview - Passed test ratio

And now a more detailed version:

	quinn	mvfst	picoquic	quic-go	aioquic	quant	quiche
stream	**100/100**	6/100	**56/100**	**95/100**	18/100	12/100	**97/100**
max	**97/100**	3/100	**47/100**	41/100	**27/100**	58/100	96/100
reset_stream	**34/100**	7/100	**61/100**	**100/100**	**24/100**	5/100	**98/100**
connection_close	**83/100**	37/100	**81/100**	**78/100**	**100/100**	40/100	**100/100**
test_unkown	**96/100**	**99/100**	99/100	**96/100**	0/100	0/100	100/100
tp_limit_newcoid	**80/100**	6/100	**50/100**	**100/100**	**24/100**	5/100	**93/100**
blocked_error	**70/100**	0/100	0/100	9/100	0/100	0/100	**100/100**
tp_error	**100/100**	**100/100**	0/100	**100/100**	0/100	0/100	0/100
tp_acticoid_error	**100/100**	0/100	0/100	0/100	0/100	**100/100**	0/100
token_error	**100/100**	**98/100**	**100/100**	**100/100**	**100/100**	**100/100**	**99/100**
no_icid	**100/100**	**100/100**	**100/100**	**100/100**	**100/100**	0/100	0/100
handshake_done_error	**99/100**	2/100	**89/100**	0/100	**86/100**	2/100	**77/100**
double_tp_error	**100/100**	**65/100**	**100/100**	**100/100**	**92/100**	**3/100**	0/100
accept_maxdata	**91/100**	**12/100**	**50/100**	**67/100**	**43/100**	**21/100**	**96/100**
stop_sending	**100/100**	4/100	**48/100**	**33/100**	**33/100**	8/100	**96/100**
ext_min_ack_delay	**78/100**	**10/100**	**38/100**	**100/100**	**26/100**	**6/100**	**98/100**

Table 1.3: Draft 29 server results detailed - Passed test ratio

Evolution from draft 18 to draft 29

We present now a more detailed version containing the number of errors per test and compare the evolution with draft 18 of the McMillan paper with draft 29. We only present the tests that McMillan already defined to see the evolution. A complete version of these results is present in the appendix.

Test	Error code	#
test_stream	require ~_generating & ~queued_non_ack(scid) -> ack_credit(scid) > 0; [5]	242
	Ran out of values for type cid	121
	require ~path_challenge_pending(dcid,f.data);	24
	bind failed: Address already in use	2
	require conn_seen(dcid) -> hi_non_probing_endpoint(dcid,dst); [10]	1
		390/1000
test_max	require stream_id_allowed(dcid,f.id); [4]	219
	Ran out of values for type cid	102
	require ~path_challenge_pending(dcid,f.data);	8
	require ~_generating & ~queued_non_ack(scid) -> ack_credit(scid) > 0; [5]	12
	require conn_total_data(the_cid) > 0;	31
	require conn_seen(dcid) -> hi_non_probing_endpoint(dcid,dst); [10]	7
	bind failed: Address already in use	1
		380/1000
test_connection_close	Ran out of values for type cid	412
	require ~_generating & ~queued_non_ack(scid) -> ack_credit(scid) > 0; [5]	53
	require ~conn_closed(scid); [2]	1
	bind failed: Address already in use	1
	require ~draining_pkt_sent(scid) & queued_close(scid);	249
	require conn_seen(dcid) -> hi_non_probing_endpoint(dcid,dst); [10]	4
	require conn_total_data(the_cid) > 0;	177
	require ~conn_closed(scid); [8]	2
		899/1000

Table 1.4: `picoquic` draft 18 - Number of errors

Test	Error code	#
test_stream	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	41
	Handshake not complete	3
		44/100
test_max	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	46
	Handshake not complete	7
		53/100
test_connection_close	Handshake not completed	2
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	17
		19/100

Table 1.5: `picoquic` draft 29 - Number of errors

Only by looking at the number of the test passed and failed we cannot conclude that `picoquic` evolves in a good way. However, when we look at the number of different errors, we can see that there are fewer errors. We can see all the corrections that have been done, for example with the migration and the `PATH_CHALLENGE` management. We conclude that we should not consider the ratio of tests that passed with those that failed. This would not be very convenient in this situation since some features are not well implemented for some implementations and occur in every test such as the migration. It can cause high differences in terms of results. For example with `aioquic`, when we disable the migration the implementation has a much better score if we only consider the passed test. We can also see that the distribution of errors changed from draft 18 to draft 29. For example, the ratio `conn_seen(dcid)->hi_non_probing_endpoint(dcid,dst)` errors increased. An explanation of it is given in the `picoquic` section.
It is important to note that these global results can induce false conclusions because all the errors don't have the same weight. Also, these results should be analyse while reading the analysis of individual traces that we have put for each implementation. Some errors are for example not critical but reduce strongly the ratio of succeeded test for one implementation. On the other side, one implementation with critical but not frequent errors has a better ratio.

What we should consider is more the type of error and the number of different errors and how they are linked to each other. For example, we know that some errors that we have are only linked to the migration so debugging this part will remove quite a lot of errors from some implementations.

General observations

We can also observe from these results that some properties are well respected by every tested implementation. For example, the test that checks the behavior of the server in case the client initiates the connection with a non-zero-length token is nearly always respected for the tested implementation.

We can also see that concerning the transport parameter and errors tests, it seems that either the feature has been correctly implemented or either not implemented at all. For example with the test `quic_server_test_tp_error`, an endpoint should

throw a TRANSPORT_PARAMETER_ERROR if it received a invalid value. Some implementations fully respect this rule while others are never sending an error. We have an exception with the mvfst implementation where it seems that the error is only sent from time to time.

With these tests, we also found an interesting question about what error should be trigger if many errors are possible for a specific situation? For example, with the test where we send unknown_frame to the server, in the case where we permit generation at any encryption level, what error should we trigger ? A FRAME_ENCODING_ERROR or a PROTOCOL_VIOLATION error ? The draft said that in such situation we should send the most "appropriate" error code in the frame that signals the error. Here again this is quite subjective and the choice taken by the implementation might influence the application over QUIC.

Finally, remember that we voluntary put an ambiguous test for the quic_server _tests _double_tp_error test where we are not sure that we should throw an error or do not complete the handshake. The results seem to confirm that the rule is ambiguous. Either the implementations didn't implement this property or they interpreted it according the two possible valid ending. Other ambiguity in the draft have been found but we explain them in detail hereafter.

aioquic

In order to prove that Ivy can be used to find and correct errors. We decide to choose an implementation, find the errors and then correct them when it is possible. Our choice for this task is aioquic since this implementation is written in Python and that is a language that we master so correcting errors is fast. You can find a complete table about the errors found at A.1.

(1) A quite simple but good implementation, there is one big problem that usually makes our general tests end into a failed state. It is that aioquic return us a PROTOCOL_VIOLATION (0xa) error indicating that the data in the PATH_CHALLENGE and PATH_RESPONSE frame are not matching. This happens during the client migration which is an important feature of QUIC.

To recall the migration concept, the client decide at some moment after the handshake to change its address, represented by a pair of IP address and port number. The connection to the server should be able to survive this event thanks to connection ID and other techniques. However for security reasons, i.e anti-spoofing source address measure, a challenge of unpredictable and encrypted data made of 8 bytes is sent by the server in a PATH_CHALLENGE frame to the client. Then the client should validate this challenge by responding with a PATH_RESPONSE frame containing the same data.

The problem is that aioquic should not throw an error since the data are in fact matching. We based our reasoning on the following RFC rule that state:

> *"A new address is considered valid when a* PATH_RESPONSE *frame is received that contains the data that was sent in a previous* PATH_CHALLENGE*. Receipt of an acknowledgment for a packet containing a* PATH_CHALLENGE *frame is not adequate validation, since the acknowledgment can be spoofed by a malicious peer.* ***Note that receipt on a different local address does not result in path validation failure****, as it might be a result of a forwarded packet (see Section 9.3.3) or misrouting. It is possible that a valid* PATH_RESPONSE *might be received in the future."*
>
> Draft 29 of QUIC section 8.5

Moreover, this problem arises more or less 50% and thus affect the result of many of our test. Next is a network trace highlighting the problem:

src	dst	Protocol	Information
4987	4443	QUIC	1274 Initial, DCID=0000000000000002, SCID=0000000000000001, PKN: 7, CRYPTO, PADDING
4443	4987		1322 Handshake, DCID=0000000000000001, SCID=0191cbd30002ee7d, PKN: 1, CRYPTO
4443	4987		1111 Handshake, DCID=0000000000000001, SCID=0191cbd30002ee7d, PKN: 2, CRYPTO
4443	4987		1322 Handshake, DCID=0000000000000001, SCID=0191cbd30002ee7d, PKN: 4, CRYPTO
4443	4987		109 Handshake, DCID=0000000000000001, SCID=0191cbd30002ee7d, PKN: 5, CRYPTO
4987	4443		158 Handshake, DCID=0191cbd30002ee7d, SCID=0000000000000001, PKN: 15, CRYPTO, PADDING
4443	4987		98 Protected Payload (KP0), DCID=0000000000000001, PKN: 6, DONE, NCI
4987	4443		1274 Protected Payload (KP0), DCID=0191cbd30002ee7d, PKN: 8, STREAM(4), PADDING
4443	4987		75 Protected Payload (KP0), DCID=0000000000000001, PKN: 7, ACK
4443	4987		82 Protected Payload (KP0), DCID=0000000000000001, PKN: 8, STREAM(4)
4443	4987		71 Protected Payload (KP0), DCID=0000000000000001, PKN: 9, PING, PADDING
4988	4443		1274 Protected Payload (KP0), DCID=0191cbd30002ee7d, PKN: 12, ACK, ACK, STREAM(8), PADDING
4443	4988		113 Protected Payload (KP0), DCID=0000000000000001, PKN: 10, ACK, DONE, PC, NCI
PATH_CHALLENGE(0a7108f0135b0b18)			
4443	4988		82 Protected Payload (KP0), DCID=0000000000000001, PKN: 11, STREAM(8)
4987	4443		1274 Protected Payload (KP0), DCID=0191cbd30002ee7d, PKN: 32, PR, ACK, PADDING
PATH_RESPONSE(0a7108f0135b0b18)			
4443	4988		106 Protected Payload (KP0), DCID=0000000000000001, PKN: 12, CC
CONNECTION_CLOSE (Transport) Error code: PROTOCOL_VIOLATION Frame Type: CONNECTION_CLOSE (Transport) (0x000000000000001c) Error code: PROTOCOL_VIOLATION (10) Frame Type: 27 Reason phrase Length: 33 Reason phrase: Response does not match challenge			

We have fixed this problem considering the first rule only and created a pull request. As you can see the problem do not arise anymore in the fixed version of `aioquic`:

src	dst	Protocol	Information
4988	4443	QUIC	1274 Initial, DCID=0000000000000001, SCID=0000000000000000, PKN: 6, CRYPTO, PADDING
4443	4988		1322 Handshake, DCID=0000000000000000, SCID=af607353e17bdead, PKN: 1, CRYPTO
4443	4988		1111 Handshake, DCID=0000000000000000, SCID=af607353e17bdead, PKN: 2, CRYPTO
4443	4988		1322 Handshake, DCID=0000000000000000, SCID=af607353e17bdead, PKN: 4, CRYPTO
4443	4988		109 Handshake, DCID=0000000000000000, SCID=af607353e17bdead, PKN: 5, CRYPTO
4988	4443		173 Handshake, DCID=af607353e17bdead, SCID=0000000000000000, PKN: 12, CRYPTO, ACK, ACK, ACK, PADDING
4443	4988		98 Protected Payload (KP0), DCID=0000000000000000, PKN: 6, DONE, NCI
4987	4443		107 Protected Payload (KP0), DCID=f5d90b5172d826f1, PKN: 13, STREAM(4), PADDING
4443	4987		78 Protected Payload (KP0), DCID=0000000000000000, PKN: 7, PC
4443	4987		75 Protected Payload (KP0), DCID=0000000000000000, PKN: 8, ACK
4443	4987		82 Protected Payload (KP0), DCID=0000000000000000, PKN: 9, STREAM(4)
4443	4987		71 Protected Payload (KP0), DCID=0000000000000000, PKN: 10, PING, PADDING
4988	4443		121 Protected Payload (KP0), DCID=af607353e17bdead, PKN: 15, PR, ACK, STREAM(12), PADDING
4443	4988		104 Protected Payload (KP0), DCID=0000000000000000, PKN: 11, ACK, DONE, NCI
4443	4988		82 Protected Payload (KP0), DCID=0000000000000000, PKN: 12, STREAM(12)
4443	4988		71 Protected Payload (KP0), DCID=0000000000000000, PKN: 13, PING, PADDING
4988	4988		108 Handshake, DCID=af607353e17bdead, SCID=0000000000000000, PKN: 29, ACK, PADDING
4443	4988		71 Protected Payload (KP0), DCID=0000000000000000, PKN: 14, PING, PADDING
4443	4988		71 Protected Payload (KP0), DCID=0000000000000000, PKN: 15, PING, PADDING
4988	4443		142 Protected Payload (KP0), DCID=f5d90b5172d826f1, PKN: 32, ACK, STREAM(16), ACK, ACK, STREAM(28), PADDING
4443	4988		91 Protected Payload (KP0), DCID=0000000000000000, PKN: 16, ACK, PING, STREAM(4)
..			

This error is probably due to a change in the draft. Before draft 20, we could only respond to the PATH_RESPONSE frame from the network path the PATH_CHALLENGE was sen. Since this feature of `aioquic` has not changed for a long time, this could explain the error.

While investigating, we found in the most recent draft new ambiguity about this subject. These rules which do not change from draft 29:

> *"Path validation succeeds when a* PATH_RESPONSE *frame is received that contains the data that was sent in a previous* PATH_CHALLENGE *frame.* ***A PATH_RESPONSE frame received on any network path validates the path on which the PATH_CHALLENGE was sent.****"*
>
> Draft 34 of QUIC section 8.2.3. [R1]

However, a new indication has been added to the draft that adds some ambiguity. The same draft state :

> *"On receiving a* PATH_CHALLENGE *frame, an endpoint MUST respond by echoing the data contained in the* PATH_CHALLENGE *frame in a* PATH_RESPONSE *frame. An endpoint MUST NOT delay transmission of a packet containing a* PATH_RESPONSE *frame unless constrained by congestion control.* ***A PATH_RESPONSE frame MUST be sent on the network path where the PATH_CHALLENGE was received****. This ensures that path validation by a peer only succeeds if the path is functional in both directions.* ***This requirement MUST NOT be enforced*** *by the endpoint that initiates path validation as that would enable an attack on migration;"*
>
> Draft 34 of QUIC section 8.2.2. [R2]

We could interpret both rules in a way that makes sense. The first rule states that you only need a valid PATH_RESPONSE and PATH_CHALLENGE whatever the path. It means that the PATH_CHALLENGE can be send on the original network path or the new one. The second rules states that the PATH_RESPONSE frame MUST be sent on the network path where the PATH_CHALLENGE was received. The combination of both rules could mean that if you send a PATH_CHALLENGE on the original path, the PATH_RESPONSE should be send from this original path and the validation is valid. The second option would be to send PATH_CHALLENGE on the migrated network path and we expect a PATH_RESPONSE that come from this migrated network path. If it is not the case, it would be invalid.

We can see one problem of the RFC. If we interpret that rule 1 (R1) as a contradiction of rule 2 (R2) and this property will be impossible to implement and to verify. If not, the rule 1 shouldn't be present in the draft as the rule 2 already imply it. The rule 1 only put more ambiguity.

While looking at the evolution of the draft, we understand why this error happens. Migration is a sensitive feature in QUIC. The rules about it changed in many drafts

between versions 19 and 28. Even in the latest draft (34), the rules changed. It is possible that while modifying the rules, some contradictions with other parts of the RFC were inserted. The migration is involved in many potential security breaches of QUIC. It is important that those rules are stabilized, unambiguous and respected.

(2) We can also see that `aioquic` is probably lacking the management of errors in the transport parameters since no errors were reported. One hypothesis of the discarding of all packets is that no check is done. A code inspection should be done to confirm this. It can also be shown that for the "ambiguous" test `quic_server_test_double_tp_error`, it decide to discard the packet.

(3) Another problem arise during migration. We did not fix it :

```
require conn_seen(dcid) -> hi_non_probing_endpoint(dcid,dst);
```

This indicates that the server didn't send packets to an address/endpoint from which the highest packet number of non-probing packets has been received.

src	dst	Protocol	Information
4988	4443	QUIC	1274 Initial, DCID=0000000000000002, SCID=0000000000000001, PKN: 7, CRYPTO, PADDING
4443	4988		1322 Handshake, DCID=0000000000000001, SCID=21500f522a60b7b2, PKN: 1, CRYPTO
4443	4988		1111 Handshake, DCID=0000000000000001, SCID=21500f522a60b7b2, PKN: 2, CRYPTO
4443	4988		1322 Handshake, DCID=0000000000000001, SCID=21500f522a60b7b2, PKN: 4, CRYPTO
4443	4988		109 Handshake, DCID=0000000000000001, SCID=21500f522a60b7b2, PKN: 5, CRYPTO
4988	4443		163 Handshake, DCID=21500f522a60b7b2, SCID=0000000000000001, PKN: 6, CRYPTO, ACK, PADDING
4443	4988		98 Protected Payload (KP0), DCID=0000000000000000, PKN: 6, DONE, NCI
4987	4443		112 Protected Payload (KP0), DCID=0f2380bbb53779ee, PKN: 1, STREAM(4), ACK, PADDING
4443	4987		78 Protected Payload (KP0), DCID=0000000000000001, PKN: 7, PC

As you can see in this example when we start the migration, we send a short header packet with a sequence number of 1 which is not erroneous since we change of connection ID. This is the random part of Ivy that chooses that in this case. Using the preceding rule, `aioquic` should have sent us the packet to the first address since we received the highest-numbered packet. One hypothesis is that `aioquic` is confused since we migrate but this is the result of the ambiguity of the draft. In Ivy, we consider this RFC rule:

> "*An endpoint only changes the address that it sends packets to in response to the highest-numbered non-probing packet. This ensures that an endpoint does not send packets to an old peer address in the case that it receives reordered packets.*"
>
> Draft 29 of QUIC section 9.3.

This rule indicates that we should send packet to the highest-numbered non-probing packet. Since there is no indication about the encryption level to consider, we used the highest-numbered non-probing packet among all possible. Maybe that since migration is only allowed after the handshake complete, then we should consider only the encryption level for the 0-RTT Protected or short header packet but this is not explicitly written.

However, in the same paragraph but presented before, we can also find the following indication:

> "*Receiving a packet from a new peer address containing a non-probing frame indicates that the peer has migrated to that address. In response to such a packet, an endpoint MUST start sending subsequent packets to the new peer address and MUST initiate path validation (Section 8.2) to verify the peer's ownership of the unvalidated address.*."
>
> Draft 29 of QUIC section 9.3.

If we use this rule then we must send responses of non-probing frame to the new peer address. Now we ask the question: Why in the same paragraph, these two rules seems to enter in contraction when we consider them in pair ? What to do if the highest-numbered non-probing packet is received on the old peer address ?

(4) One last note about `aioquic` is that we observed that the `qlogs` are usually not working, or at least for the version we tested. For example, we have launch 30 local tests and get only two `.qlog` files. This indicates again that this implementation is lacking of some features including basic ones.

`quinn`

(1) The first error is again linked to migration. We can see that we get a requirement that fails indicating that we already have a pending `path_challenge` frame with the same data/challenge. The Ivy rule that failed and the trace for more detail:

```
require ~path_challenge_pending(dcid,f.data)
```

src	dst	time	Protocol	Information
4987	4443		QUIC	1274 Initial, DCID=0000000000000000, SCID=0000000000000001, PKN:0, CRYPTO, PADDING
4443	4987			1242 Handshake, DCID=0000000000000001, SCID=e09f058956664da8, PKN:0, CRYPTO, PADDING
4443	4987			96 Protected (KP0) , DCID=0000000000000001, PKN:0, NCI
4987	4443			158 Handshake, DCID=e09f058956664da8, SCID=0000000000000001, PKN:0, CRYPTO, PADDING
4443	4987			71 Protected (KP0) , DCID=0000000000000001, PKN:1, DONE, PADDING
4988	4443			1274 Protected (KP0) , DCID=e09f058956664da8, PKN:0, ACK, ACK, PADDING
4443	4987			1242 Protected (KP0) , DCID=0000000000000001, PKN:2, PC, PADDING
PATH_CHALLENGE (00fe42f00ca1b5a9), PADDING				
4988	4443			1274 Protected (KP0) , DCID=e09f058956664da8, PKN:16, PR, PADDING
4443	4988	37.38		1242 Protected (KP0) , DCID=0000000000000001, PKN:3, ACK, PC, PADDING
ACK, PATH_CHALLENGE (03a32cf4a177000f), PADDING				
4443	4988	40.58		1242 Protected (KP0) , DCID=0000000000000001, PKN:4, PING, ACK, PC, PADDING
PING, ACK, PATH_CHALLENGE (03a32cf4a177000f), PADDING				

We can see that the packet is retransmitted since the payload of the packet is the same with only the packet number changing. Moreover, the time interval between the two packets seems to confirm this idea. This long time is not wanted and is due to the computations of Ivy.

This goes in contradiction with the section "13.3 Retransmission of Information" of the draft 29. It is written that:

> "*A liveness or path validation check using* PATH_CHALLENGE *frames is sent periodically until a matching* PATH_RESPONSE *frame is received or until there is no remaining need for liveness or path validation checking.* PATH_CHALLENGE *frames* ***include a different payload each time they are sent.***"
>
> Draft 29 of QUIC section 13.3

However, there is also the following general property in the retransmission telling us that:

> "*New frames and packets are used to carry information that is determined to have been lost.* ***In general,*** *information is sent again when a packet containing that information is determined to be lost and sending ceases when a packet containing that information is acknowledged.*"
>
> Draft 29 of QUIC section 13.3

We can conclude that when the implementation noticed that the PATH_CHALLENGE was lost, it tries to retransmit the frame. Either the developer forgets this feature and the general rule is applied, either this is a programming mistake. We are currently working on this problem since it seems reachable for us.

(2) We also have the failure of the following requirement in the QUIC packet specification:

```
require ~ _generating  ∧ ~ queued_non_ack(scid)
        -> ack_credit(scid)> 0; # [5]
```

Let's explain this requirement by part:

- ~ queued_non_ack(scid): queued_non_ack(C) is a relation that indicates that one or more of queued frames at aid C is not an ACK frame. Wwith the negation rule we have that we shouldn't have an ACK frame for C.
- ack_credit(scid)> 0: The relation ack_credit(scid) is the number of ack-eliciting packet minus the number non-ack-eliciting packet which are any packet containing other frames than ACK, PADDING, CONNECTION_CLOSE

The final relation indicates that the number of non-ack-eliciting packets must not be greater than the number of ack-eliciting packets.
We can find this in the RFC:

> *"Since packets containing only ACK frames are not congestion controlled, an endpoint MUST NOT send more than one such packet in response to receiving an ack-eliciting packet."*
>
> Draft 29 of QUIC section 13.2.1

We can see in this network trace that we receive an ACK for the same packet twice in a very short time interval:

src	dst	Protocol	Information
4987	4443	QUIC	1274 Initial, DCID=0000000000000002, SCID=0000000000000001, PKN: 7, CRYPTO, PADDING
4443	4987		1242 Handshake, DCID=0000000000000001, SCID=b61bd296f77cd819, PKN: 0, CRYPTO, PADDING
4443	4987		96 Protected Payload (KP0), DCID=0000000000000001, PKN: 0, NCI
4987	4443		163 Handshake, DCID=b61bd296f77cd819, SCID=0000000000000001, PKN: 6, CRYPTO, ACK, PADDING
4443	4987		71 Protected Payload (KP0), DCID=0000000000000001, PKN: 1, DONE, PADDING
4987	4443		117 Protected Payload (KP0), DCID=dd2e767c11e3e999, PKN: 2, ACK, ACK, STREAM(4), PADDING
4443	4987		73 Protected Payload (KP0), DCID=0000000000000001, PKN: 2, ACK
4443	4987		76 Protected Payload (KP0), DCID=0000000000000001, PKN: 3, ACK, MS
4443	4987		116 Protected Payload (KP0), DCID=0000000000000001, PKN: 4, ACK, STREAM(4)
4987	4443		117 Protected Payload (KP0), DCID=b61bd296f77cd819, PKN: 10, ACK, ACK, STREAM(4), PADDING
4443	4987		75 Protected Payload (KP0), DCID=0000000000000001, PKN: 5, ACK
4987	4443		107 Protected Payload (KP0), DCID=dd2e767c11e3e999, PKN: 16, STREAM(8), PADDING
4443	4987		77 Protected Payload (KP0), DCID=0000000000000001, PKN: 6, ACK
4443	4987		80 Protected Payload (KP0), DCID=0000000000000001, PKN: 7, ACK, MS
4443	4987		120 Protected Payload (KP0), DCID=0000000000000001, PKN: 8, ACK, STREAM(8)
4987	4443		112 Protected Payload (KP0), DCID=b61bd296f77cd819, PKN: 12, STREAM(8), ACK, PADDING
4443	4987		77 Protected Payload (KP0), DCID=0000000000000001, PKN: 9, ACK
4987	4443		107 Protected Payload (KP0), DCID=dd2e767c11e3e999, PKN: 32, STREAM(12), PADDING
4443	4987		79 Protected Payload (KP0), DCID=0000000000000001, PKN: 10, ACK
ACK(LA: 32, AD: 0, ACK Cnt:3 ...)			
4443	4987		82 Protected Payload (KP0), DCID=0000000000000001, PKN: 11, ACK, MS
ACK(LA: 32, AD: 0, ACK Cnt:3 ...)			
4443	4987		122 Protected Payload (KP0), DCID=0000000000000001, PKN: 12, ACK, STREAM(12)
ACK(LA: 32, AD: 0, ACK Cnt:3 ...)			

(3) We pass the test about the management of unknown frames all the time so this feature is well implemented. However, we can also see that sometimes the handshake does not finish and no error is reported. It happens only when we allow generation of `unknown_frame` at any time in the execution. This is normal that the handshake does not finish since this frame is not allowed at this time. However, we can ask the question about which rules of the specification to respect in such a case? Is it up to the wishes of the programmers ?

(4) The last error we get is about the following requirement:

```
require stream_id_allowed(dcid,f.id);
```

This error happens when we test the `RESET_STREAM` frame. It means that the stream ID to be reset defined in the `RESET_STREAM`/`STREAM_STREAM` frame must not exceed the maximum stream ID allowed for the corresponding stream type (or that the stream ID has not been reset).

src	dst	Protocol	Information
4988	4443	QUIC	1274 Initial, DCID=0000000000000001, SCID=0000000000000000, PKN: 19, CRYPTO, PADDING
4443	4988		1242 Handshake, DCID=0000000000000000, SCID=70e611f95304c122, PKN: 0, CRYPTO, PADDING
4443	4988		96 Protected (KP0) , DCID=0000000000000001, PKN:0, NCI
4988	4443		158 Handshake, DCID=70e611f95304c122, SCID=0000000000000000, PKN: 7, CRYPTO, PADDING
4443	4988		71 Protected Payload (KP0), DCID=0000000000000000, PKN: 1, DONE, PADDING
4987	4443		92 Protected Payload (KP0), DCID=afee30bdbe4a630d, PKN: 6, ACK, PADDING
4443	4988		1242 Protected Payload (KP0), DCID=0000000000000000, PKN: 2, PC, PADDING
4988	4443		103 Protected Payload (KP0), DCID=afee30bdbe4a630d, PKN: 16, PR, RS(4), PADDING
4443	4988		75 Protected Payload (KP0), DCID=0000000000000000, PKN: 3, ACK
4443	4988		81 Protected Payload (KP0), DCID=0000000000000000, PKN: 4, ACK, MS, STREAM(4)
MS_BIDI(100); STREAM(4): Length = 0; Stream Data: <MISSING>			

In this case, we can see that stream 4 is reset but `quinn` still sends us a `STREAM` frame with ID 4 and without data. It notices the frame since just before it acknowledged

the packet (PKN = 16).

It seems that there is some ambiguity in the stream FSMs. First of all, even if we suppose that the RFC wanted to have only one initial state, this caused a problem for the one-to-one matching between the sending FSM and the receiving FSM states. This can cause ambiguities. The Figure 1.1 show an example of it.

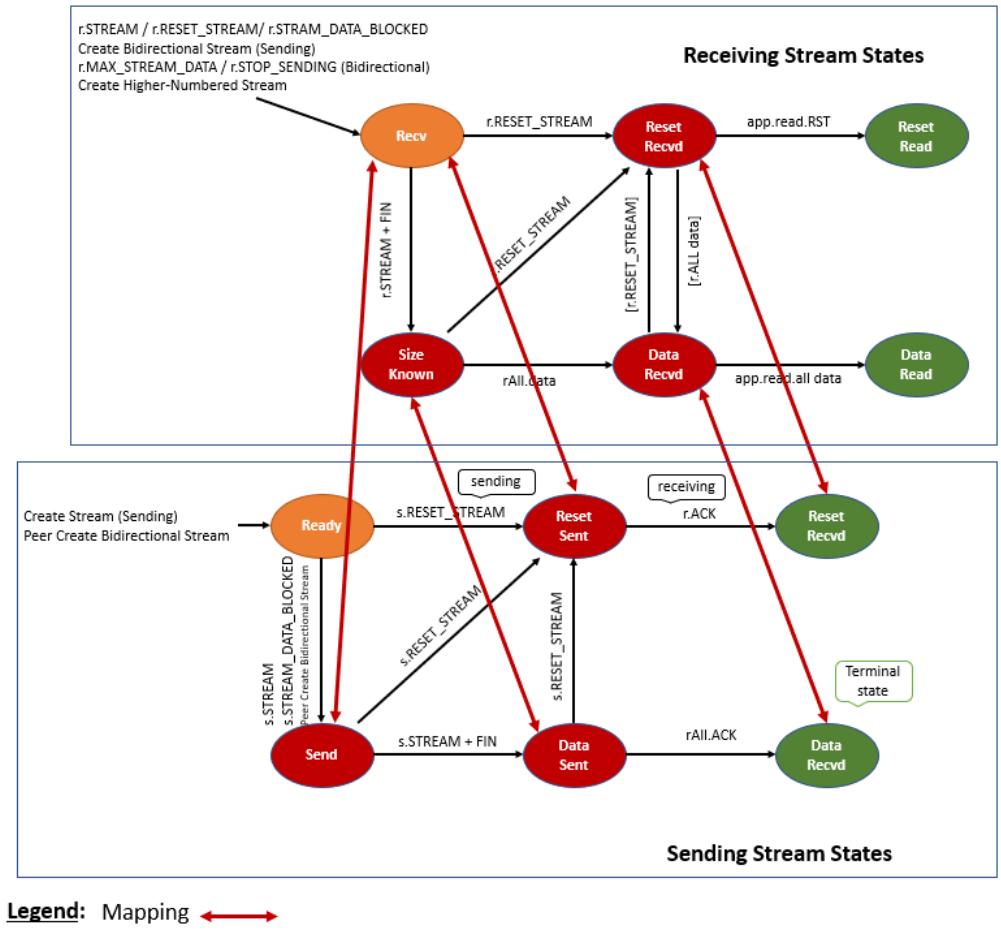


Figure 1.1: Mapping between FSM

When we consider the sending FSM and the "Reset Sent" state, we can see that there is only one outgoing state to "Reset Recvd". It is a terminal state. We consider the stream as closed.

When the stream is terminated, and since according to the draft we can only open a stream of the same type in increasing order, we cannot receive another bidirectional stream with 4 as stream ID.

For the receiving FSM, when we receive a `RESET_STREAM` frame, we enter the "Reset Recvd" state. From this state, there is still the possibility to enter the "Data Recvd" state. It is not the case from the sending FSM.

The draft states for RESET_STREAM frame:

> *"An endpoint uses a* RESET_STREAM *frame (type=0x04) to abruptly terminate the* ***sending part of a stream****. After sending a* RESET_STREAM*, an endpoint ceases transmission and retransmission of* STREAM *frames on the identified stream. A receiver of* RESET_STREAM *can discard any data that it already received on that stream."*
>
> Draft 29 of QUIC section 19.4. [A1]

Now we can see how the FSM are used with the following statement:

> *"Unidirectional streams use the applicable state machine directly.* ***Bidirectional streams use both state machines****. For the most part, the use of these state machines is the same whether the stream is unidirectional or bidirectional. The conditions for opening a stream are slightly* ***more complex*** *for a bidirectional stream because the opening of either the sending or receiving side causes the stream to open in both directions."*
>
> Draft 29 of QUIC section 3. [A2]

To link these rules, with A1 we see that the sending part of a stream is terminated. Then A2, says that bidirectional streams use both state machines. Should we consider that each endpoint uses its own version of the FSM since the one-to-one mapping is difficult? Or should we consider that the FSMs are shared between the endpoints?

If each endpoint uses its own FMSs then which sending FSM to terminate? Receiver or sender? This solution seems possible if we consider the possibility to allow the other endpoint to send data. It would be like if we return in a unidirectional case somehow.

If the endpoints use a shared version of the FSMs, then the termination of the sending part of the FSM should affect both endpoints. This would mean that no data can be sent on the stream. The eventual STREAM frame in flight could still be received. It seems also valid.

(5) One last note is that quinn implements well the errors in the transport parameter part. For all of the concerned tests, it always returns an error even when it is not mandatory, i.e for the double transport parameter error.

mvfst

First of all, we do not manage to complete the handshake at each run. When we manage to complete it, we still have various errors as we can see in A.3.

(1) The errors are usually similar among the tests. Let's first explain the errors code we get in the connection_close frame.

- APPLICATION_ERROR (0xc): indicating that the application or application protocol caused the connection to be closed.

This error seems to appears quite a lot, from 10% to 25% of the time depending on the test. After investigation, the error message says "Migration disable". It comes from our Ivy logs. The implementation logs tell us:

```
4472 RecordLayer.cpp:82] Received handshake message Finished
4472 StateMachine-inl.h:41] Transition from ExpectingFinished to
    AcceptingData
...
633 LogQuicStats.h:60 server onPacketDropped
    reason=PEER_ADDRESS_CHANGE
633 QuicTransportBase.cpp:1870 onNetworkData Migration disabled
633 EchoHandler.h:53 Socket "error=Invalid migration"
633 LogQuicStats.h:91 server onConnectionClose reason=NONE
```

This is unexpected since according to our Ivy logs, we didn't receive the `disable_active_migration` transport parameter and the logs of the implementation only tell us that this is not valid when we perform it. Moreover, the `HANDSHAKE_DONE` frame has been received so the migration is allowed. A hypothesis is that our "Hello Retry" problem, which we describe in the third point (3), perturbs our specification.

src	dst	Protocol	Information
4987	4443	QUIC	1274 Initial, DCID=0000000000000002, SCID=0000000000000001, PKN: 10, CRYPTO, PADDING
4443	4987		1294 Initial, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480956, CRYPTO, ACK, PADDING
4443	4987		1294 Initial, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480958, CRYPTO, ACK, PADDING
4443	4987		1294 Initial, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480961, CRYPTO, ACK, PADDING
4443	4987		1294 Initial, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480963, CRYPTO, ACK, PADDING
4443	4987		1294 Initial, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480964, CRYPTO, ACK, PADDING
4443	4987		1274 Initial, DCID=400000d42cb02248, SCID=0000000000000001, PKN: 27, CRYPTO, PADDING
4443	4987		1294 Initial, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480965, CRYPTO, ACK, PADDING
4443	4987		814 Handshake, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480956, CRYPTO
Extension: quic_transport_parameters (drafts version) (len=89)			
Parameter: original_destination_connection_id (len=8)			
Parameter: initial_max_stream_data_bidi_local (len=4) 66560			
Parameter: initial_max_stream_data_bidi_remote (len=4) 66560			
Parameter: initial_max_stream_data_uni (len=4) 66560			
Parameter: initial_max_data (len=4) 1048576			
Parameter: initial_max_streams_bidi (len=2) 2048			
Parameter: initial_max_streams_uni (len=2) 2048			
Parameter: max_idle_timeout (len=4) 60000 ms			
Parameter: ack_delay_exponent (len=1)			
Parameter: max_udp_payload_size (len=2) 1500			
Parameter: stateless_reset_token (len=16)			
Parameter: facebook_partial_reliability (len=1)			
Parameter: initial_source_connection_id (len=8)			
4443	4987		1294 Initial, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480967, CRYPTO, ACK, PADDING
4443	4987		1294 Initial, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480968, ACK, PADDING
4987	4443		150 Handshake, DCID=400000d42cb02248, SCID=0000000000000001, PKN: 7, CRYPTO, ACK, PADDING
4443	4987		89 Handshake, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480957, ACK
4443	4987		71 Protected Payload (KP0), DCID=0000000000000001, PKN: 5480956, DONE
4988	4443		95 Protected Payload (KP0), DCID=400000d42cb02248, PKN: 19, ACK, PADDING
4443	4987		92 Protected Payload (KP0), DCID=0000000000000001, PKN: 5480957, CC
4443	4987		105 Handshake, DCID=0000000000000001, SCID=400000d42cb02248, PKN: 5480958, CC

We can see that it throw us an error when we start migration just after receiving the `HANDSHAKE_DONE`. It consider that we breaks the specification. In the draft it is said that we cannot perform the migration before the handshake is confirm:

> "*The design of QUIC relies on endpoints retaining a stable address for the duration of the handshake. An endpoint MUST NOT initiate connection migration before the handshake is confirmed, as defined in section 4.1.2 of [QUIC-TLS].*"
>
> Draft 29 of QUIC section 9.

However we can argue with the following requirement in QUIC-TLS for the client view:

> "*In this document, the TLS handshake is considered confirmed at the server when the handshake completes. At the client, the handshake is considered confirmed when a* HANDSHAKE_DONE *frame is received.*"
>
> Draft 29 of QUIC-TLS section 4.1.2.

So from, the client point of view, we can consider the handshake as confirmed either when we received the HANDSHAKE_DONE frame or either when we ACK the packet containing this frame.

For the server, the handshake is confirmed when the handshake is complete. It happens when the TLS stack has both sent a "Finished" message and verified the peer's "Finished" message which is the case here (at least for the sending part).

> "*In this document, the TLS handshake is considered* ***complete*** *when the TLS stack has reported that the handshake is complete. This happens when the TLS stack has both sent a Finished message and verified the peer's Finished message. Verifying the peer's Finished provides the endpoints with an assurance that previous handshake messages have not been modified. Note that the handshake does not complete at both endpoints simultaneously. Consequently, any requirement that is based on the completion [also confirmation for server] of the handshake depends on the perspective of the endpoint in question.*"
>
> Draft 29 of QUIC-TLS section 4.1.1.

(2) This "error" is about the unknown_frame test. We have that mvfst SIGTERM with the following error message in its logs (so without return error):

```
I0417 23:58:05.578828 12458 EchoServer.h:90] Echo server started at:
    127.0.0.1:4443
E0417 23:58:11.153787 12463 EchoHandler.h:53] Socket error=Frame
    format error
*** Aborted at 1618703891 (Unix time, try 'date -d @1618703891') ***
*** Signal 15 (SIGTERM) (0x30a7) received by PID 12458 (pthread TID
    0x7f762ccb2ec0) (linux TID 12458) (maybe from PID 12455, UID 0)
    (code: 0), stack trace: ***
(error retrieving stack trace)
```

This happens only when we allow the generation unknown_frame for any encryption level (in this example, Initial). In the other case, mvfst behave well by reporting a FRAME_ENCODING_ERROR. When we are in the Initial encryption level it can either discard the packet or return an error. When we are in the Handshake encryption level it must return an error. So according to these rules, mvfst behave well and decide to drop packet when we are in the Initial encryption level.

> "*The payload of an Initial packet includes a* CRYPTO *frame (or frames) containing a cryptographic handshake message,* ACK *frames, or both.* PING, PADDING, *and* CONNECTION_CLOSE *frames of type 0x1c are also permitted. An endpoint that receives an Initial packet containing other frames can* ***either*** *discard the packet as spurious or treat it as a connection error.*"
>
> Draft 29 of QUIC section 17.2.2.

> "*The payload of this packet [Handshake] contains* CRYPTO *frames and could contain* PING, PADDING, *or* ACK *frames. Handshake packets MAY contain* CONNECTION_CLOSE *frames of type 0x1c. Endpoints MUST treat receipt of Handshake packets with other frames as a connection error.*"
>
> Draft 29 of QUIC section 17.2.2.

(3) Concerning the problem of the handshake, it seems that `mvfst` try to respond to a connection request with four QUIC/TLS packets of type "Hello Retry Request" and then send a normal "Server Hello" at the fifth trial. It could be the reason why we manage to do the handshake and sometimes not.

	time	src	dst	Protocol	Information
4379	125.357	localhost	localhost	QUIC	1274 Initial, DCID=0000000000000001, SCID=0000000000000000, PKN:0
4381	125.358				1294 Initial, DCID=0000000000000000, SCID=40000087630fcf40, PKN:12755779
4387	125.471				1294 Initial, DCID=0000000000000000, SCID=40000087630fcf40, PKN:12755781
4393	125.682				1294 Initial, DCID=0000000000000000, SCID=40000087630fcf40, PKN:12755783
TLSv1.3 Record Layer: Handshake Protocol: Hello Retry Request					
4394	125.683	localhost	localhost	QUIC	1294 Initial, DCID=0000000000000000, SCID=40000087630fcf40, PKN:12755784
4506	126.090				1274 Initial, DCID=40000087630fcf40, SCID=0000000000000000, PKN:16
4507	126.091				1294 Initial, DCID=0000000000000000, SCID=40000087630fcf40, PKN:12755785
TLSv1.3 Record Layer: Handshake Protocol: Server Hello					

The cost of implementing this feature would be to update the serializer/deserializer of the TLS part and the associated Ivy modelization. This is the "easy" part. We would also need to update the PicoTLS API interface with Ivy and we think that with some deeper analysis of PicoTLS and eventually also by analyzing other QUIC implementation using PicoTLS, we could solve the problem.

(4) It can also be shown that for the "ambiguous" test `quic_server_test_double_tp_error`, the handshake do not complete. it either decides to discard the packet or stop sending us packets for other reasons. No information in the logs seems to tell us that it detects the double transport parameter.

(5) It can also be shown that for the "ambiguous" test `quic_server_test_token_error`, we get a INTERNAL_ERROR from the implementation in a CONNECTION_CLOSE frame:

```
I0418 10:41:27.839520 15506 EchoServer.h:90] Echo server started at:
    127.0.0.1:4443
I0418 10:41:46.003042 15510 EchoHandler.h:48] Socket closed
*** Aborted at 1618742506 (Unix time, try 'date -d @1618742506') ***
*** Signal 15 (SIGTERM) (0x3c8f) received by PID 15506 (pthread TID
    0x7f30a8656ec0) (linux TID 15506) (maybe from PID 15503, UID 0)
    (code: 0), stack trace: ***
(error retrieving stack trace)
```

But the draft says that we should :

> "*Token Length: A variable-length integer specifying the length of the Token field, in bytes. This value is zero if no token is present. Initial packets sent by the server MUST set the Token Length field to zero; clients that receive an Initial packet with a non-zero The token Length field MUST either discard the packet or generate a connection error of type* `PROTOCOL_VIOLATION`."
>
> Draft 29 of QUIC section 17.2.2.

But since in the other tests of the batch, it behaves well, we are not sure that this is specifically our test that causes the internal error. It might happen in "any" case.

(6) For the test where we voluntarily send `HANDSHAKE_DONE` frame but we are not allowed. `mvfst` has "choose" the strategy to discard the connection since we usually don't finish the handshake. But this is still unexpected since two times we get the test that passed. After investigation, we discover that usually we were performing migration and the deliverable of the frame at the same time and `mvfst` decide to first report the invalid migration. When we disable the migration, it reacts as expected.

(7) For the error in general, either it is fully implemented either nothing is done.

`picoquic`

We can find the detailed version of the results at A.4.

(1) We mainly have one property that fails:

```
require conn_seen(dcid) -> hi_non_probing_endpoint(dcid,dst);
```

We already got this error for `aioquic`. The interpretation of this error is the same as the one in the `aioquic` section. This error is appearing more often than in draft 18.

It is probably a good example of how the global interpretation of results can be misleading. The code was not fixed between the two drafts but it is more appearing now. It is probably because other errors were fixed between the two drafts. `picoquic` fixed its implementation of the migration between the two drafts and it could be the reason why this error happens more often. We can suppose that the migration errors of the previous draft are "hiding" this error. As a recall, Ivy stops on the first error seen.

src	dst	Protocol	**Information**
4988	4443	QUIC	1274 Initial, DCID=0000000000000002, SCID=0000000000000001, PKN: 7, CRYPTO, PADDING
4443	4988		1294 Handshake, DCID=0000000000000001, SCID=340db0bf519f3482, PKN: 0, CRYPTO
4443	4988		506 Protected Payload (KP0), DCID=0000000000000001, PKN: 0, NT, NCI, PADDING
4443	4988		1482 Protected Payload (KP0), DCID=0000000000000001, PKN: 1, PING, PADDING
4988	4443		158 Handshake, DCID=1a7780ee8e9a53b8, SCID=0000000000000001, PKN: 10, CRYPTO, PADDING
4443	4988		225 Protected Payload (KP0), DCID=0000000000000001, PKN: 2, NT, NCI, PADDING
4443	4988		97 Protected Payload (KP0), DCID=0000000000000001, PKN: 3, DONE, PADDING
4443	4988		1442 Protected Payload (KP0), DCID=0000000000000001, PKN: 4, PING, PADDING
4443	4988		225 Protected Payload (KP0), DCID=0000000000000001, PKN: 5, NT, NCI, PADDING

4443	4988	97 Protected Payload (KP0), DCID=0000000000000001, PKN: 6, DONE, PADDING
4443	4988	225 Protected Payload (KP0), DCID=0000000000000001, PKN: 7, NT, NCI, PADDING
4988	4443	167 Protected Payload (KP0), DCID=1a7780ee8e9a53b8, **PKN: 15**, STREAM(4), ACK, ACK, ACK, STREAM(8), STREAM(12), ACK, PADDING
4443	4988	1249 Protected Payload (KP0), DCID=0000000000000001, PKN: 8, ACK, STREAM(4), STREAM(8), STREAM(12), PADDING
4988	4443	237 Protected Payload (KP0), DCID=340db0bf519f3482, PKN: 1, STREAM(4), ACK, ACK, ACK, STREAM(8), STREAM(12), ACK, STREAM(16), STREAM(24), ACK, STREAM(20), ACK, PADDING
4443	4988	1249 Protected Payload (KP0), DCID=0000000000000001, PKN: 9, ACK, STREAM(16), STREAM(20), STREAM(24), PADDING
4443	4988	97 Protected Payload (KP0), DCID=0000000000000001, PKN: 10, DONE, ACK, PADDING
4443	4988	225 Protected Payload (KP0), DCID=0000000000000001, PKN: 11, NT, NCI, ACK, PADDING
4987	4443	132 Protected Payload (KP0), DCID=1a7780ee8e9a53b8, PKN: 4, ACK, STREAM(32), STREAM(28), PADDING
4443	4987	1294 Protected Payload (KP0), DCID=0000000000000001, PKN: 12, PC, ACK, PADDING

We can see here that the packet with 15 as the sequence number is the highest non-probing frame that we send. `Picoquic` should send packets on port 4987 until we send him a non-probing frame with a higher packet number than 15.

(3) For the implementation of the error management of the transport parameter, we can conclude that `picoquic` either not implemented not, either choose to ignore the bad value in the transport parameter. If it ignores it, this does not conform to the draft.

(4) Looking at this result, we can also see that `picoquic` implement a good strategy for unknown frames since it only sends `PROTOCOL_VIOLATION` (`0xa`) errors or doesn't complete the handshake when we send an unknown frame in the wrong encryption level. In other cases, it sends a `FRAME_ENCODING_ERROR` as the draft specified in section [12.4.]

`quic-go`

The table referring its result is A.5.

(1) The following rule is not respected and this is the only implementation that produce it:

```
require f.fin <-> (stream_app_data_finished(dcid,f.id)
  & offset+f.length = stream_app_data_end(dcid,f.id));
```

This indicates that when a `STREAM` frame has the `fin` bit set to 1, then we should have the corresponding stream finished and the total stream offset must match. And inversely if the `fin` bit is set to 0. This does not concern retransmission.

> *"The final size of a stream is always signaled to the recipient. The final size is the sum of the Offset and Length fields of a STREAM frame with a FIN flag, noting that these fields might be implicit."*
>
> Draft 29 of QUIC section 4.4.

In our example, we can see that `quic-go` retransmit all the data from the `STREAM(4)` even if we acknowledged the preceding stream which is not allowed and quite strange.

We know that this is retransmission since we get exactly the same content in the frames. No useful information in the logs too.

src	dst	time	Protocol	Information
4988	4443		QUIC	1274 Initial, DCID=0000000000000002, SCID=0000000000000001, PKN: 7, CRYPTO, PADDING
4443	4988			1294 Handshake, DCID=0000000000000000, SCID=e87f172d, PKN: 0, CRYPTO
4443	4988			228 Handshake, DCID=0000000000000000, SCID=e87f172d, PKN: 1, CRYPTO
4443	4988			1294 Initial, DCID=0000000000000000, SCID=e87f172d, PKN: 1, PADDING, CRYPTO
4443	4988			1294 Initial, DCID=0000000000000000, SCID=e87f172d, PKN: 2, PADDING, CRYPTO
4988	4443			154 Handshake, DCID=e87f172d, SCID=0000000000000000, PKN: 13, CRYPTO, PADDING
4443	4988			1127 Handshake, DCID=0000000000000000, SCID=e87f172d, PKN: 2, ACK, CRYPTO
4443	4988			228 Handshake, DCID=0000000000000000, SCID=e87f172d, PKN: 3, CRYPTO
4443	4988			1394 Protected Payload (KP0), DCID=0000000000000000, PKN: 0, PADDING, PING
4443	4988			319 Protected Payload (KP0), DCID=0000000000000000, PKN: 1, DONE, NT, CRYPTO
4443	4988			319 Protected Payload (KP0), DCID=0000000000000000, PKN: 3, DONE, NT, CRYPTO
4443	4988			319 Protected Payload (KP0), DCID=0000000000000000, PKN: 4, DONE, NT, CRYPTO
4443	4988			1344 Protected Payload (KP0), DCID=0000000000000000, PKN: 5, PADDING, PING
4988	4443			103 Protected Payload (KP0), DCID=e87f172d, PKN: 15, STREAM(4), PADDING
4443	4988			74 Protected Payload (KP0), DCID=0000000000000000, PKN: 6, ACK
4443	4988			1294 Protected Payload (KP0), DCID=0000000000000000, PKN: 7, STREAM(4)
4443	4988			1294 Protected Payload (KP0), DCID=0000000000000000, PKN: 8, STREAM(4)
4443	4988			1294 Protected Payload (KP0), DCID=0000000000000000, PKN: 9, STREAM(4)
4443	4988			1294 Protected Payload (KP0), DCID=0000000000000000, PKN: 10, STREAM(4)
4443	4988			187 Protected Payload (KP0), DCID=0000000000000000, PKN: 11, STREAM(4)
4443	4988			73 Protected Payload (KP0), DCID=0000000000000000, PKN: 12, STREAM(4)
4443	4988			319 Protected Payload (KP0), DCID=0000000000000000, PKN: 14, DONE, NT, CRYPTO
4443	4988			1294 Protected Payload (KP0), DCID=0000000000000000, PKN: 15, STREAM(4)
4443	4988			1319 Protected Payload (KP0), DCID=0000000000000000, PKN: 16, PADDING, PING
4988	4443	41.63		88 Protected Payload (KP0), DCID=e87f172d, PKN: 32, ACK, PADDING
4988	4443	43.3848		113 Protected Payload (KP0), DCID=e87f172d, PKN: 36, STREAM(12), ACK, ACK, PADDING
4443	4988	43.3851		1303 Protected Payload (KP0), DCID=0000000000000000, PKN: 17, ACK, STREAM(4)
4443	4988	43.3852		1294 Protected Payload (KP0), DCID=0000000000000000, PKN: 18, STREAM(4)
4443	4988	43.3852		187 Protected Payload (KP0), DCID=0000000000000000, PKN: 19, STREAM(4)
4443	4988	43.3855		73 Protected Payload (KP0), DCID=0000000000000000, PKN: 20, STREAM(4)
4443	4988			1319 Protected Payload (KP0), DCID=0000000000000000, PKN: 21, STREAM(12)
4443	4988			1319 Protected Payload (KP0), DCID=0000000000000000, PKN: 22, STREAM(12)
4443	4988			1319 Protected Payload (KP0), DCID=0000000000000000, PKN: 23, STREAM(12)

After an analysis, we are in presence of a clear false positive. In Ivy, we already acknowledged the last `STREAM(4)` send by `quic-go` but more or less at the same time the retransmission timer of `quic-go` expires and then retransmit some `STREAM(4)`. Since in the Ivy state, this stream has already been processed and that we consider the stream finished and acknowledged, we report an error. We can see a limitation of Ivy, it took 2 seconds to process all messages of `quic-go` which causes retransmission. One solution to this problem would be to configure an implementation to have a longer retransmission timer.

(2) Except that we can still conclude that `quic-go` behave well for transport parameter properties and the management of unknown frame. We can also see that for the "ambiguous" test `quic_server_test_double_tp_error`, it decides to send a `TRANSPORT_PARAMETER_ERROR`.

(3) The fact that none of our tests about the `HANDSHAKE_DONE` frame passed might indicate that `quic-go` has still some trouble for this part. It can explain the problem of connection we faced. For this test, `quic-go` usually returns us a `FRAME_ENCODING_ERROR` which is unexpected since this the only implementation to do that. The specification states:

> "*A* `HANDSHAKE_DONE` *frame can only be sent by the server. Servers MUST NOT send a* `HANDSHAKE_DONE` *frame before completing the handshake. A server MUST treat receipt of a* `HANDSHAKE_DONE` *frame as a connection error of type* `PROTOCOL_VIOLATION`"
>
> Draft 29 of QUIC section 19.20.

Moreover the error message indicates that the feature is not well implemented. The message says that we cannot send this frame with the "Handshake" encryption which is true but it was intentional in our case. The most important thing is that it doesn't detect that the frame has been sent by a client.

src	dst	Protocol	Information
4988	4443	QUIC	1274 Initial, DCID=0000000000000000, SCID=0000000000000001, PKN: 0, CRYPTO, PADDING
4443	4988		1294 Handshake, DCID=0000000000000001, SCID=6d0df88cb463f2a1, PKN: 0, CRYPTO
4443	4988		465 Handshake, DCID=0000000000000001, SCID=6d0df88cb463f2a1, PKN: 1, CRYPTO
4443	4988		213 Initial, DCID=0000000000000001, SCID=6d0df88cb463f2a1, PKN: 1, CRYPTO
4443	4988		213 Initial, DCID=0000000000000001, SCID=6d0df88cb463f2a1, PKN: 2, CRYPTO
4443	4988		1118 Handshake, DCID=0000000000000001, SCID=6d0df88cb463f2a1, PKN: 2, CRYPTO
4443	4988		465 Handshake, DCID=0000000000000001, SCID=6d0df88cb463f2a1, PKN: 3, CRYPTO
4988	4443		109 Handshake, DCID=6d0df88cb463f2a1, SCID=0000000000000001, PKN: 0, DONE, DONE, DONE, DONE, DONE, DONE, PADDING
4443	4988		240 Protected Payload (KP0), DCID=0000000000000001, PKN: 0, CC
CONNECTION_CLOSE (Transport) Error code: FRAME_ENCODING_ERROR Error code: FRAME_ENCODING_ERROR (7) Frame Type: 30 Reason phrase Length: 60 Reason phrase: HandshakeDoneFrame not allowed at encryption level Handshake			

`quant`

We have numerous number of error with this implementation as we can see in the results table A.6.

(1) This trigger the failure of a requirement in the QUIC packet specification, i.e the same than for the `quinn` implementation:

```
require ~ _generating  ∧ ~ queued_non_ack(scid)
        -> ack_credit(scid)> 0; # [5]
```

src	dst	Protocol	**Information**
4988	4443	QUIC	1274 Initial, DCID=0000000000000002, SCID=0000000000000001, PKN: 7, CRYPTO, PADDING
4443	4988		1242 Initial, DCID=0000000000000001, SCID=5015dece, PKN: 0, ACK, PADDING, CRYPTO
4443	4988		1294 Handshake, DCID=0000000000000001, SCID=5015dece, PKN: 0, PADDING, CRYPTO
4443	4988		354 Handshake, DCID=0000000000000001, SCID=5015dece, PKN: 1, PADDING, CRYPTO
4988	4443		154 Handshake, DCID=5015dece, SCID=0000000000000001, PKN: 3, CRYPTO, PADDING
4443	4988		105 Protected Payload (KP0), DCID=0000000000000001, PKN: 0, DONE, NT
4988	4443		1274 Protected Payload (KP0), DCID=5015dece, PKN: 16, ACK, PADDING
4443	4988		74 Protected Payload (KP0), DCID=0000000000000001, PKN: 1, ACK
4988	4443		1274 Protected Payload (KP0), DCID=5015dece, PKN: 32, ACK, PADDING
4443	4988		73 Protected Payload (KP0), DCID=0000000000000001, PKN: 2, ACK
4988	4443		1274 Protected Payload (KP0), DCID=5015dece, PKN: 33, ACK, PADDING
4443	4988		73 Protected Payload (KP0), DCID=0000000000000001, PKN: 3, ACK
ACK (LA: 33, ACK Delay: 37, ACK Range Count: 0, First ACK range:0)			

We can see that `quant` acknowledged `ACK` frame which does not respect the property. Maybe also that some problems come from the fact that the encryption cipher for level 3 is not set for the first packet of this type received.

(2) Another result is for the test `quic_server_test_tp_error` where we voluntary send an invalid value for the `ack_delay_exponent` transport parameter. In theory we should get a `TRANSPORT_PARAMETER_ERROR` but instead `quant` get a runtime error and crash with the following error message:

```
[0m[41m[0m[35m err_close[30m  [34m conn.c:1691 [0mack_delay_exponent
    16040 invalid
/quic/quant/lib/src/tls.c:1465:9: runtime error: null pointer passed
    as argument 7, which is declared to never be null
```

We also have a similar behavior when we send the same transport parameter twice or for the test where we omit the original connection ID transport parameter. We suppose there is none or little coverage of the transport parameter value check from `quant`.

(3) We can also see that for the test where we test the `connection_close` frame, we have many errors. Among them an error of invalid state:

```
require conn_draining(scid)-> ~draining_pkt_sent(scid)&
                    queued_close(scid);
```

This rule indicates that if the sender is in the draining state, then we consider it as a "draining packet". This implies no retransmission of these packets and the presence of a `CONNECTION_CLOSE` frame these the packet. In this case, we can see that we have retransmission of the packet.

src	dst	Protocol	Information
4988	4443	QUIC	1274 Initial, DCID=0000000000000001, SCID=0000000000000000, PKN: 7, CRYPTO, PADDING
4443	4988		1242 Initial, DCID=0000000000000000, SCID=d2676493, PKN: 0, ACK, PADDING, CRYPTO
4443	4988		1294 Handshake, DCID=0000000000000000, SCID=d2676493, PKN: 0, PADDING, CRYPTO
4443	4988		351 Handshake, DCID=0000000000000000, SCID=d2676493, PKN: 1, PADDING, CRYPTO
4988	4443		115 Handshake, DCID=d2676493, SCID=0000000000000000, PKN: 18, ACK, CC, PADDING
4443	4988		89 Handshake, DCID=0000000000000000, SCID=d2676493, PKN: 2, ACK, CC
4988	4443		104 Handshake, DCID=d2676493, SCID=0000000000000000, PKN: 31, ACK, PADDING
4443	4988		91 Handshake, DCID=0000000000000000, SCID=d2676493, PKN: 3, ACK, CC

This correspond to the following RFC rule:

> *"After receiving a* `CONNECTION_CLOSE` *frame, endpoints enter the draining state. An endpoint that receives a* `CONNECTION_CLOSE` *frame MAY send a single packet containing a* `CONNECTION_CLOSE` *frame before entering the draining state, using a* `CONNECTION_CLOSE` *frame and a* `NO_ERROR` *code if appropriate. An endpoint MUST NOT send further packets, which could result in a constant exchange of* `CONNECTION_CLOSE` *frames until the closing period on either peer ended"*
>
> Draft 29 of QUIC section 10.3.

(4) Finally, we also have a problem of `PATH_CHALLENGE` where we receive the same data for the challenge. This is not allowed and this occurs even if we only allow one migration. As you can see the behavior is unexpected:

src	dst	Protocol	Information
4443	4987	QUIC	71 Protected Payload (KP0), DCID=0000000000000001, PKN: 2, PING, PADDING
4988	4443		1274 Protected Payload (KP0), DCID=71155229, PKN: 0, ACK, STREAM(24), STREAM(16), STREAM(8), PADDING
4988	4443		1274 Protected Payload (KP0), DCID=c38284e7, PKN: 4, STREAM(28), ACK, ACK, STREAM(12), PADDING
4443	4988		120 Protected Payload (KP0), DCID=0000000000000001, PKN: 3, ACK, DONE, NT, PC
PATH_CHALLENGE (1dc4add05cbb8c5f)			
4443	4988		110 Protected Payload (KP0), DCID=0000000000000001, PKN: 4, PC, PADDING, STREAM(28)
PATH_CHALLENGE (1dc4add05cbb8c5f)			
4443	4988		110 Protected Payload (KP0), DCID=0000000000000001, PKN: 5, PC, PADDING, STREAM(12)
PATH_CHALLENGE (1dc4add05cbb8c5f)			
4988	4443		1274 Protected Payload (KP0), DCID=c38284e7, PKN: 5, STREAM(28), ACK, ACK, STREAM(12), PADDING
4443	4988		83 Protected Payload (KP0), DCID=0000000000000001, PKN: 6, ACK, PC
PATH_CHALLENGE (1dc4add05cbb8c5f)			

`quiche`

Except for the following, we consider `quiche` as a mature implementation without much-detected error. You can have this observation in the global results A.7.

(1) As the first observation, we can see that `quiche` choose, voluntary or not, to not react when an endpoint initiate connection with an invalid transport parameter value. We usually get a timeout where in most case it should return us an error:

```
Finished dev [unoptimized + debuginfo] target(s) in 0.12s
Running `tools/apps/target/debug/quiche-server --cert
    tools/apps/src/bin/cert.crt --key tools/apps/src/bin/cert.key
    --no-retry --dump-packets /results/dump.txt --listen
    '127.0.0.1:4443'`
[2021-04-18T18:05:39.038882591Z ERROR quiche_server]
    d21987e22b078c1d8b4a32d0308c321f recv failed:
    InvalidTransportParam
```

But according to the draft,

> *"An endpoint MUST treat receipt of a transport parameter with an invalid value as a connection error of type* `TRANSPORT_PARAMETER_ERROR`*."*
>
> Draft 29 of QUIC section 7.4.

We suppose that this is voluntary since `quiche` is made by Cloudflare. It is a big player in the industry and in security. To be sure we should ask them directly. Our interpretation is that they probably do nothing since it could reveal useful information that an attacker could eventually use. For example if an attacker try to do fuzzing with these transport parameter to crash the system, then never reporting errors is way to hide information about the potential vulnerabilities. We based this idea on a mitigation technique that can be used against SQL injection and which consist to not disclose more information than necessary since for this attack, we should guess the architecture of the database with some kind of fuzzing. However, this way of proceeding can be problematic if an application needs these error messages. This is only a supposition. The RFC is also not respected anyways.

Review: supposition allowed? can we put this on the discussion part of the conclusion

(2) The other tests we launch were successful except few cases where we encountered the same error than other implementations about the `ACK`, see A.7. However, we are able to test only unidirectional cases for unknown reasons. It is probably due to a wrong configuration:

src	dst	Protocol	Information
4988	4443	QUIC	1274 Initial, DCID=0000000000000000, SCID=0000000000000001, PKN: 0, CRYPTO, PADDING
4443	4988		226 Initial, DCID=0000000000000001, SCID=bd56b6665d7d4ad4a2e00aa1075ba8b5, PKN: 0, ACK, CRYPTO
4443	4988		1238 Handshake, DCID=0000000000000001, SCID=bd56b6665d7d4ad4a2e00aa1075ba8b5, PKN: 0, CRYPTO
4443	4988		287 Handshake, DCID=0000000000000001, SCID=bd56b6665d7d4ad4a2e00aa1075ba8b5, PKN: 1, CRYPTO
4988	4443		150 Handshake, DCID=bd56b6665d7d4ad4a2e00aa1075ba8b5, SCID=0000000000000001, PKN: 6, CRYPTO, PADDING
4443	4988		97 Handshake, DCID=0000000000000001, SCID=bd56b6665d7d4ad4a2e00aa1075ba8b5, PKN: 2, ACK
4988	4443		72 Protected Payload (KP0), DCID=0000000000000001, PKN: 0, DONE, PADDING
4443	4988		1274 Protected Payload (KP0), DCID=bd56b6665d7d4ad4a2e00aa1075ba8b5, PKN: 12, ACK, PADDING
4988	4443		1274 Protected Payload (KP0), DCID=bd56b6665d7d4ad4a2e00aa1075ba8b5, PKN: 4, STREAM(8), STREAM(12), STREAM(20), PADDING
4443	4988		78 Protected Payload (KP0), DCID=0000000000000001, PKN: 1, ACK
4988	4443		1274 Protected Payload (KP0), DCID=bd56b6665d7d4ad4a2e00aa1075ba8b5, PKN: 0, ACK, STREAM(4), PADDING
			...

`lsquic`

As said before, we get a TLS internal error.

src	dst	Protocol	Information
4988	4443	QUIC	1274 Initial, DCID=0000000000000000, SCID=0000000000000001, PKN: 0, CRYPTO, PADDING
4443	4988		108 Initial, DCID=0000000000000001, SCID=ea9727ea9e27fa35, PKN: 0, ACK, CC
CONNECTION_CLOSE (Transport) Error code: CRYPTO_ERROR (Internal Error) Frame Type: CONNECTION_CLOSE (Transport) (0x1c) Error code: CRYPTO_ERROR (336) TLS Alert Description: Internal Error (80) Frame Type: 0 Reason phrase Length: 12 Reason phrase: TLS alert 80			

The logs not helping us too:

```
12:22:15.657 [DEBUG] [QUIC:0000000000000000] event: decrypted packet 0
12:22:15.657 [DEBUG] [QUIC:0000000000000000] event: packet in: 0, type: Initial, size:
                     1232; ecn: 0, spin: 0; path: 0
12:22:15.657 [DEBUG] [QUIC:0000000000000000] event: CRYPTO frame in: level 0; offset 0;
                     size 234
12:22:15.658 [DEBUG] [QUIC:0000000000000000] event: PADDING frame in of 948 bytes
12:22:15.658 [DEBUG] engine: decref conn 0000000000000000, 'HT' -> 'H'
12:22:15.658 [DEBUG] [QUIC:0000000000000000] event: generated ACK frame: [0-0]
12:22:15.658 [DEBUG] engine: incref conn 0000000000000000, 'H' -> 'HO'
12:22:15.658 [DEBUG] engine: incref conn 0000000000000000, 'HO' -> 'CHO'
12:22:15.658 [DEBUG] engine: decref conn 0000000000000000, 'CHO' -> 'CO'
12:22:15.658 [DEBUG] engine: batched packet 0 for connection 0000000000000000
12:22:15.658 [DEBUG] engine: batched all outgoing packets for mini conn 0000000000000000
12:22:15.658 [DEBUG] engine: packets out returned 1 (out of 1)
12:22:15.658 [DEBUG] [QUIC:0000000000000000] event: sent packet 0, type Initial, crypto:
                      clear, size 66, frame types: ACK CONNECTION_CLOSE, ecn: 0, spin: 0;
                      kp: 0, path: 0, flags: 98829
12:22:15.658 [DEBUG] engine: decref conn 0000000000000000, 'CO' -> 'C'
12:22:15.658 [DEBUG] engine: send_packets_out: sent 1 packet
12:22:15.658 [DEBUG] engine: decref conn 0000000000000000, 'C' -> ''
```

We still present the implementation since we have some results for the clients.

1.3.2 Summary

Let's summarize. The first point to highlight is that we were not able to test all implementation. It is the case for the server test with `lsquic`. When we look at other testing tools used for QUIC, this seems normal. We also decide to focus on the type of error rather than the number of errors since it could be misinterpreted or lead to a false conclusion.

We can also see that the tested implementation by McMillan evolve in a good way. Less errors are reported than for the draft 18. For the streams management, most errors that were found in draft 18 disappear.

We manage to find different types of errors such as internal errors or requirements in the draft that are not followed. For the internal errors, we spotted mostly memory errors from the implementations. As expected, these errors were principally present in an implementation written in a low-level programming language.

When we considered the management of the errors by the implementation, we could observe that most of the time either the features are well implemented, either nothing was done. One representative implementation is `picoquic` that react well for some scenarios such as the omission of the `original_destination_connection_id` transport parameter. Not for those where we put an invalid value, for example with the `active_connection_id_limit` transport parameter where the value is not checked.

There is also the fact that some implementations send error code with wrong message. However these problems can only be found with a post-analysis of the results and not directly with Ivy.

We can also see that some implementations seem to voluntarily disable some features concerning the errors in the transport parameter. `quiche` is the perfect example since for all these tests it never sends an error. One hypothesis is that this is voluntary since `quiche` is made by Cloudflare which is a big player in the security industry. A hypothesis is that they probably do nothing since it could reveal useful information that an attacker could eventually use.

Another important point to highlight is the migration of clients. We have seen that there is a lot of ambiguity in the draft for this part and this is reflected in the different implementations. One of the biggest and most important ambiguities is about the validation of the `PATH_CHALLENGE`. Just to remember, here are the requirements that are ambiguous:

> *"Path validation succeeds when a* PATH_RESPONSE *frame is received that contains the data that was sent in a previous* PATH_CHALLENGE *frame.* ***A PATH_RESPONSE frame received on any network path validates the path on which the PATH_CHALLENGE was sent.****"*
>
> Draft 34 of QUIC section 8.2.3. [R1]

> *"On receiving a* PATH_CHALLENGE *frame, an endpoint MUST respond by echoing the data contained in the* PATH_CHALLENGE *frame in a* PATH_RESPONSE *frame. An endpoint MUST NOT delay transmission of a packet containing a* PATH_RESPONSE *frame unless constrained by congestion control.* ***A PATH_RESPONSE frame MUST be sent on the network path where the PATH_CHALLENGE was received****. This ensures that path validation by a peer only succeeds if the path is functional in both directions.* ***This requirement MUST NOT be enforced*** *by the endpoint that initiates path validation as that would enable an attack on migration;"*
>
> Draft 34 of QUIC section 8.2.2. [R2]

We can see that are sure with these statements if we can or cannot validate a migration when we receive a PATH_RESPONSE on a different network path than the PATH_CHALLENGE was sent. This is reflected in the tested implementation where some consider that this is not allowed and others that this allowed. This is also reflected in our modelization of QUIC with Ivy since we still have to do a choice, as for the QUIC developer, on what rules to enforce.

Some RFC rules should be more precise about some situations and changed. For example,

> *"In this document, the TLS handshake is considered confirmed at the server when the handshake completes. At the client, the handshake is considered confirmed when a* HANDSHAKE_DONE *frame is received."*
>
> Draft 29 of QUIC-TLS section 4.1.2.

So from, the client's point of view, we can consider the handshake as confirmed either when we received the HANDSHAKE_DONE frame or either when we ACK the packet containing this frame. We could replace this rule with this one for example :

> *"In this document, the TLS handshake is considered confirmed at the server when the handshake completes. At the client, the handshake is considered confirmed when a* HANDSHAKE_DONE *frame is received* ***and acknowledge****."*
>
> Draft 29 of QUIC-TLS section 4.1.2. modified

However, even this solution is far from perfect since even if we ACK the frame we are not sure that both client and server will be synchronized on their state. However, for the migration since we would need to wait until we acknowledged the

`HANDSHAKE_DONE` before starting the migration, we should respect the RFC.

We also found that there are sometimes no indications in the RFC on what to do in some situations. Here again we can clearly see that the implementations have to make a choice by themselves. But we think that this is not good since the RFC should at least consider the situation and then let the choice to the developer. A good example of that is what to do if someone retransmits `HANDSHAKE_DONE` frame "too much" times? Should we interpret that as some kind of attack? Should we report an error? Should we do nothing since nothing is specified? Some implementations like `picoquic` decide to do nothing, some of them report errors like `quinn`.

Some implementations also have small problems with their ACK management since sometimes for no apparent reason, some of them acknowledge the same packet twice in a very short period of time.

Finally, some false positives are found. It is mainly due to the performance problem of Ivy. A good example is with `quic-go` where a small-time interval and packet batch cause the detection of a false positive. This means that we should never use the result provided by Ivy directly. A post-analysis of the result is always required.

Review: dire dans cette partie que les resultats concordent +- avec ceux de mcmillan ?

Review: Le summary n'est pas totalement fini car il faut encore la partie client

Review: On compte mettre des liens vers differentes trace et example présenter nous n'avons pas encore eu le temps de faire ca

Bibliography

[1] J. Iyengar and M. Thomson, "QUIC: A UDP-Based Multiplexed and Secure Transport," Internet Engineering Task Force, Internet-Draft draft-ietf-quic-transport-29, work in Progress. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-ietf-quic-transport-29

[2] [Online]. Available: https://github.com/ElNiak/QUIC-Ivy/tree/quic_29/doc/examples/quic

[3] Quicwg, "quicwg/base-drafts." [Online]. Available: https://github.com/quicwg/base-drafts/wiki/Implementations

[4] [Online]. Available: https://github.com/private-octopus/picoquic

[5] [Online]. Available: https://github.com/h2o/picotls

[6] [Online]. Available: https://github.com/lucas-clemente/quic-go

[7] [Online]. Available: https://github.com/aiortc/aioquic

[8] [Online]. Available: https://github.com/NTAP/quant/tree/29

[9] [Online]. Available: https://github.com/facebookincubator/mvfst

[10] [Online]. Available: https://github.com/litespeedtech/lsquic

[11] [Online]. Available: https://boringssl.googlesource.com/boringssl/

[12] [Online]. Available: https://github.com/quinn-rs/quinn

Appendix A

Results tables

In the following part, we present you the detailed version of the errors we found. Since we launch 100 iterations per test, we give the number of time each error appears. This allow to see the error frequency.

After analysing the data, we conclude that we should not consider the ratio of test that passed with those that failed. This would not be very convenient in this situation since some features are not well implemented for some implementation such as the migration. It can cause high differences in term of results. For example with `aioquic`, when we disable the migration, the implementation have much better score if we only consider the passed test. So this is not relevant.

It is more important to consider the type of error and their distribution. Also, some errors are linked between each other. For example, we know that some errors that we have are only linked to the migration. If an implementation correct this part, it will remove multiple errors at once.

A.1 aioquic

Table A.1: aioquic server tests errors

Test	Error code	#
test_stream	Handshake not completed	5
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	26
	frame.connection$_c lose$: {err_code : 0xa};	50
	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	1
		82/100
test_max	Handshake not completed	6
	frame.connection$_c lose$: {err_code : 0xa};	42
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	24
	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	1
		73/100
test_reset_stream	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	15
	frame.connection$_c lose$: {err_code : 0xa};	52
	Handshake not completed	9
		76/100
test_connection_close	Handshake not completed	2
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	9
	require is_no_error	11
		22/100
test_unkown	frame.connection_close:{err_code:0xa}	100
		100/100
test_tp_error	Handshake not completed	86
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	11
	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	3
		100/100
test_tp_acticoid_error	Handshake not completed	**100/100**
test_token_error	Handshake not completed	**100/100**
test_no_icid	require is_transport_parameter	94
	Handshake not completed	6
		100/100
test_handshake_done_error	require is_protocol_violation	2
	Handshake not completed	7
	SegmentationFault 139	5
		14/100
test_double_tp_error	Handshake not completed	92
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	3
	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	4
		100/100
test_blocked_streams_maxstream_error	Handshake not completed	83
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	7
	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	3
	require is_frame_encoding_error | is_stream_limit_error;	7
		100/100
test_accept_maxdata	require conn_total_data(the_cid) >0;	2
	Handshake not completed	2
	frame.connection_close:{err_code:0xa}	26
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	24
	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	3
		57/100
test_ext_min_ack_delay	Handshake not completed	7
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	29
	frame.connection_close:{err_code:0xa}	37
	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	1
		74/100

A.2 quinn

Table A.2: quinn server test errors

Test	Error code	#
test_stream	Handshake not completed	22
		22/100
test_max	Handshake not completed	1
	require stream_id_allowed(dcid,f.id);	22
		23/100
test_reset_stream	require stream_id_allowed(dcid,f.id);	57
	Handshake not completed	9
		66/100
test_connection_close		**0/100**
test_unkown	Handshake not completed	4
		4/100
test_tp_error		**0/100**
test_tp_acticoid_error		**0/100**
test_token_error		**0/100**
test_no_icid		**0/100**
test_handshake_done_error	Segmentation Fault 134	**1/100**
test_double_tp_error		**0/100**
test_blocked_streams_maxstream_error	Handshake not completed	30
		30/100
test_accept_maxdata	Handshake not completed	7
	require stream_id_allowed(dcid,f.id);	8
	require conn_total_data(the_cid) >0;	2
		17/100
test_ext_min_ack_delay	Handshake not completed	22
		22/100

A.3 mvfst

Table A.3: mvfst server tests errors

Test	Error code	#
test_stream	Handshake not completed	25
	frame.connection_close:{err_code:0xc}	62
	Local Error	7
		94/100
test_max	Local Error	1
	frame.connection_close:{err_code:0xc}	76
	Handshake not completed	20
		97/100
test_reset_stream	Local Error	3
	frame.connection_close:{err_code:0xc}	69
	Handshake not completed	21
		93/100
test_connection_close	Handshake not completed	16
	require is_no_error	46
	require connected(dcid) & connected_to(dcid) = scid;	1
		63/100
test_unkown	frame.connection_close:{err_code:0xc}	1
		1/100
test_tp_error		**0/100**
test_tp_acticoid_error	Handshake not completed	31
	frame.connection_close:{err_code:0xc}	69
		100/100
test_token_error	frame.connection_close:{err_code:0x1}	**1/100**
test_no_icid		**0/100**
test_handshake_done_error	Segmentation Fault 139	97
	Handshake not completed	1
		98/100
test_double_tp_error	Handshake not completed	64
	frame.connection_close:{err_code:0xc}	35
		100/100
test_blocked_streams_maxstream_error	Handshake not completed	62
	frame.connection_close:{err_code:0xc}	34
	Local Error	4
		100/100
test_accept_maxdata	Local Error	4
	frame.connection_close:{err_code:0xc}	34
	Handshake not completed	62
		74/100
test_ext_min_ack_delay	Local Error	5
	frame.connection_close:{err_code:0xc}	64
	Handshake not completed	21
		90/100

A.4 picoquic

Table A.4: picoquic server tests errors

Test	Error code	#
test_stream	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	41
	Handshake not complete	3
		44/100
test_max	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	46
	Handshake not complete	7
		53/100
test_reset_stream	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	36
	Handshake not completed	3
		39/100
test_connection_close	Handshake not completed	2
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	17
		19/100
test_unkown	require is_frame_encoding_error	1
		1/100
test_tp_error	Handshake not completed	81
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	17
	require is_transport_parameter_error;	2
		100/100
test_tp_acticoid_error	Handshake not completed	96
	require is_transport_parameter_error	4
		100/100
test_token_error		**0/100**
test_no_icid		**0/100**
test_handshake_done_error	require is_protocol_violation	1
	Handshake not complete	10
		11/100
test_double_tp_error		**0/100**
test_blocked_streams_maxstream_error	Handshake not completed	79
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	13
	require is_frame_encoding_error | is_stream_limit_error;	8
		100/100
test_accept_maxdata	require conn_total_data(the_cid) >0;	6
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	41
	Handshake not completed	3
		50/100
test_ext_min_ack_delay	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	34
	require f.seq_num >last_ack_freq_seq(scid);	19
	Handshake not completed	9
		62/100

A.5 quic-go

Table A.5: quic-go server tests errors

Test	Error code	#
test_stream	require f.fin <-> (stream_app_data_finished(dcid,f.id) ...	**5/100**
test_max	server_return_code(1)+test_completed	2
	Handshake not completed	59
	require f.fin <-> (stream_app_data_finished(dcid,f.id) ...	2
		63/100
test_reset_stream		**0/100**
test_connection_close	Handshake not completed	**57/100**
test_unkown	require is_frame_encoding_error;	3
	Handshake not completed	1
		4/100
test_tp_error		**0/100**
test_tp_acticoid_error	Handshake not completed	99
	Local Error	1
		100/100
test_token_error	Local Error	**0/100**
test_no_icid		**0/100**
test_handshake_done_error	frame.connection_close:{err_code:0x7}	92
	Handshake not completed	8
		100/100
test_double_tp_error		**0/100**
test_blocked_streams_maxstream_error	Handshake not completed	**100/100**
test_accept_maxdata	Handshake not completed	13
	require f.fin <-> (stream_app_data_finished(dcid,f.id) ...	4
	server_return_code(1)+test_completed	16
		33/100
test_ext_min_ack_delay		**0/100**

A.6 quant

Table A.6: quant server tests errors

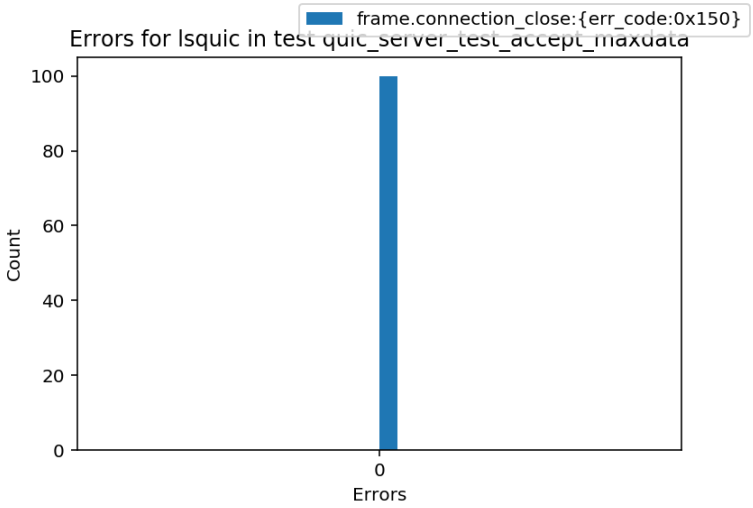
Test	Error code	#
test_stream	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	4
	require ~path_challenge_pending(dcid,f.data);	31
	Handshake not completed	19
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	34
		88/100
test_max	Handshake not completed	20
	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	1
	require ~path_challenge_pending(dcid,f.data);	31
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	27
		79/100
test_reset_stream	Handshake not completed	24
	require ~path_challenge_pending(dcid,f.data);	36
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	35
		95/100
test_connection_close	Handshake not completed	7
	require ~conn_closed(scid);	1
	Local Error	12
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	18
	require ~draining_pkt_sent(scid) & queued_close(scid);	5
	require ~path_challenge_pending(dcid,f.data);	17
		60/100
test_unkown	require is_frame_encoding_error;	36
	Handshake not completed	58
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	1
	Local Error	5
		100/100
test_tp_error	server_return_code(1)+timeout	100
		100/100
test_tp_acticoid_error		**0/100**
test_token_error		**0/100**
test_no_icid	server_return_code(1)+timeout'	100
		100/100
test_handshake_done_error	Handshake not completed	69
	require is_protocol_violation	29
		98/100
test_double_tp_error	server_return_code(1)+timeout	100
		100/100
test_blocked_streams_maxstream_error	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	11
	Handshake not completed	46
	require ~path_challenge_pending(dcid,f.data);	28
	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	15
		100/100
test_accept_maxdata	Handshake not completed	2
	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	1
	require ~path_challenge_pending(dcid,f.data);	58
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	17
	require conn_total_data(the_cid) >0;	1
		79/100
test_ext_min_ack_delay	require ~path_challenge_pending(dcid,f.data);	34
	Handshake not completed	25
	require conn_seen(dcid) ->hi_non_probing_endpoint(dcid,dst);	35

A.7 quiche

Table A.7: quiche server tests errors

Test	Error code	#
test_stream	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	2
	Local Error	1
		3/100
test_max	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	**4/100**
test_reset_stream	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	**2/100**
test_connection_close		**0/100**
test_unkown		**0/100**
test_tp_error	Local Error	**100/100**
test_tp_acticoid_error	Local Error	**100/100**
test_token_error	Local Error	**1/100**
test_no_icid	Local Error	**100/100**
test_handshake_done_error	require is_protocol_violation;	11
	Local Error	12
		33/100
test_double_tp_error	Local Error	**100/100**
test_blocked_streams_maxstream_error		**0/100**
test_accept_maxdata	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	**4/100**
test_ext_min_ack_delay	require ~_generating & ~queued_non_ack(scid) ->ack_credit(scid) >0;	**2/100**

A.8 lsquic


frame.connection_close:{err_code:0x150}
Errors for lsquic in test quic_server_test_accept_maxdata
100
80
60
40
20
0
Count
0
Errors